



Graph Coloring via GNN

Alessandro Calderoni

Dipartimento di Fisica, Università La Sapienza, Piazzale Aldo Moro 5, Roma

Abstract

Questo elaborato presenta un approccio basato sull'uso di Graph Neural Network per risolvere il problema della colorazione di grafi tramite ottimizzazione differenziabile. Il modello associa a ogni nodo una rappresentazione continua dei colori, addestrando la rete attraverso la minimizzazione di una funzione di loss ispirata al modello di Potts, progettata per penalizzare l'assegnazione del medesimo colore a nodi adiacenti, con un aggiuntivo termine entropico di regolarizzazione. Applicando la pipeline a due grafi, è stato stimato con successo il numero cromatico di ciascuna struttura, individuando il valore minimo di colori necessario per garantire una soluzione esatta.

Introduzione

Il problema della colorazione dei grafi è una sfida classica nel campo dell'ottimizzazione combinatoria. Dato un grafo non orientato $G = (V, E)$ e un numero intero di colori disponibili c , lo scopo è determinare se sia possibile assegnare a ogni nodo $i \in V$ uno tra gli c colori, garantendo che due nodi adiacenti non abbiano mai la stessa assegnazione. Formalmente, per ogni arco $(i, j) \in E$, deve valere il vincolo discreto: $\text{color}(i) \neq \text{color}(j)$. Questo progetto esplora un approccio basato sulla riformulazione della colorazione dei grafi come un problema di ottimizzazione differenziabile. Per fare ciò, si utilizza una Graph Neural Network (GNN) che, invece di assegnare direttamente un colore rigido, produce per ogni nodo una rappresentazione continua dell'assegnazione del colore. Il cuore del metodo è la funzione di costo utilizzata per addestrare la rete. Questa funzione è costruita come una versione differenziabile dell'energia del modello fisico di Potts, la quale agisce penalizzando gli archi che collegano nodi a cui la rete tende ad assegnare probabilità elevate per il medesimo colore. L'ottimizzazione è quindi specifica per la singola istanza: per ogni grafo di input fornito, i parametri della GNN vengono ottimizzati al fine di minimizzare l'energia di Potts di quella specifica struttura. L'obiettivo finale è trovare l'assegnazione ottimale che minimizzi il numero di archi in conflitto e, ripetendo la procedura per diversi valori di c , stimare il numero cromatico del grafo stesso $\chi(G)$, cioè in numero c di colori minimo per colorare il grafo senza conflitti.

Strategia

Per rendere il problema trattabile tramite algoritmi di ottimizzazione basati sul gradiente, il vincolo discreto di assegnazione del colore viene rilassato: per ogni nodo $i \in V$, la GNN riceve una rappresentazione iniziale e, attraverso strati di **message passing**, calcola un vettore di caratteristiche che viene successivamente convertito in una distribuzione di probabilità sugli c colori disponibili tramite

l'applicazione della funzione **Softmax**:

$$\mathbf{p}_i = (p_{i1}, p_{i2}, \dots, p_{ic}), \quad \text{con } p_{ik} = \frac{e^{z_{ik}}}{\sum_{j=1}^c e^{z_{ij}}} \quad (1)$$

dove z_{ik} rappresenta il logit di output della rete per il nodo i relativamente al colore k , e p_{ik} esprime la confidenza o probabilità associata all'assegnazione di quel colore.

Il cuore dell'ottimizzazione risiede nella definizione di una funzione di costo differenziabile ispirata all'hamiltoniana antiferromagnetica del **modello di Potts**. L'obiettivo è penalizzare gli archi del grafo i cui nodi adiacenti presentano distribuzioni di colore sovrapposte. Al fine di poter applicare il metodo in modo consistente e confrontabile a grafi di dimensioni differenti, la loss viene normalizzata rispetto al numero totale di archi $|E|$:

$$\mathcal{L}_{\text{Potts}} = \frac{1}{|E|} \sum_{(i,j) \in E} \mathbf{p}_i \cdot \mathbf{p}_j = \frac{1}{|E|} \sum_{(i,j) \in E} \sum_{k=1}^c p_{ik} p_{jk} \quad (2)$$

Questa quantità assume valori prossimi allo zero quando nodi adiacenti hanno probabilità concentrate su colori distinti, mentre cresce significativamente in presenza di sovrapposizioni.

Per evitare soluzioni patologiche in cui la rete converga verso distribuzioni uniformi ed eccessivamente sfuocate (assegnazioni "morbide" stabili ma prive di significato combinatorio), viene introdotta una regolarizzazione entropica. Tale termine penalizza l'incertezza della rete, favorendo configurazioni nette e vicine a vettori **one-hot**:

$$H = -\frac{1}{|V|} \sum_{i \in V} \sum_{k=1}^c p_{ik} \log p_{ik} \quad (3)$$

La loss totale ottimizzata risulta quindi dalla combinazione lineare dei due contributi, pesati tramite un iperparametro di regolarizzazione λ :

$$\mathcal{L}_{\text{tot}} = \mathcal{L}_{\text{Potts}} + \lambda H \quad (4)$$

Un'altra possibile strada per la regolarizzazione è quella di introdurre una temperatura nella **Softmax** di output. Il panorama energetico definito dalla loss totale è altamente



non convesso e caratterizzato da molteplici minimi locali. Per mitigare la dipendenza dall'inizializzazione casuale dei pesi della rete, l'ottimizzazione viene ripetuta più volte adottando una strategia di **multi-start** con seed casuali differenti per ogni ripetizione. Durante il processo, vengono monitorati sia l'andamento della loss continua sia il numero effettivo di conflitti discreti ad ogni epoca.

Al termine della fase di training, la transizione dalla rappresentazione continua a quella discreta avviene applicando l'operatore **argmax** sul vettore delle probabilità di ciascun nodo:

$$\text{color}(i) = \arg \max_{1 \leq k \leq c} p_{ik} \quad (5)$$

La validità della soluzione viene quantificata calcolando il numero di conflitti residui $N_{\text{conflicts}}$ (archi i cui nodi adiacenti condividono lo stesso colore discreto) e il corrispondente errore normalizzato $\epsilon = N_{\text{conflicts}}/|E|$. Una colorazione è considerata valida se il numero di conflitti è pari a zero.

Qualora l'ottimizzazione converga a una soluzione sub-ottimale con un numero esiguo di conflitti residui, viene applicata una procedura di **post-processing** locale di tipo **greedy** per risolvere le ultime criticità: si identificano gli archi in conflitto e si tenta di variare localmente il colore di uno dei nodi coinvolti, accettando il cambiamento solo se questo elimina o riduce il numero totale di conflitti residui.

Per determinare il numero cromatico dei grafi assegnati, viene eseguita una scansione sul numero di colori disponibili ($c = \max_{i \in V} \text{degree}(i) + 1, \dots, 2$). Il valore minimo di c per il quale la pipeline strutturata trova una soluzione a zero conflitti viene registrato come la stima finale del numero cromatico del grafo.

Architettura

Si è scelto l'impiego di un'architettura semplice ma ben controllata per agevolare l'ottimizzazione dell'energia di Potts. La GNN sviluppata si basa su strati convoluzionali di tipo **SAGEConv**. Questo approccio permette un **message passing** efficiente, aggregando le informazioni dal vicinato di ciascun nodo.

L'architettura elabora le feature iniziali dei nodi attraverso due strati nascosti, seguiti da un livello di classificazione lineare e una normalizzazione finale. Nello specifico, la pipeline sequenziale è così strutturata:

1. **Input (Embedding Iniziale):** Ad ogni nodo $i \in V$ viene assegnato un vettore di embedding iniziale $\mathbf{x}_i^{(0)}$ di dimensione d_{in} . Tali embedding possono essere generati casualmente o derivati da proprietà topologiche del grafo e costituiscono l'input del primo strato. Nel nostro caso sono stati usati vettori **one-hot** corrispondenti all'indice del nodo:

$$\mathbf{x}_i^{(0)} = \mathbf{e}_i, \quad (6)$$

dove $\mathbf{e}_i$ indica l'usuale vettore della base canonica di $\mathbb{R}^{|V|}$. La scelta dell'embedding corretto è fondamentale: provando con vettori **one-hot** corrispondenti al grado del nodo, la rete non riusciva a colorare correttamente. Un'altra scelta, invece funzionante, è quella di usare vettori appartenenti a $\mathbb{R}^c$, generati casualmente (rispetto a vettori **one-hot** casuali in $\mathbb{R}^c$, con cui si rischia di finire in minimi locali della loss, da cui la rete non riesce ad uscire).

2. **Primo Strato SAGEConv:** L'embedding iniziale viene processato da un layer **SAGEConv** che mappa le feature in uno spazio latente a 64 dimensioni (scelta abbastanza arbitraria). L'operazione aggrega le feature del nodo con quelle dei suoi vicini diretti. L'output passa poi attraverso una funzione di attivazione non lineare **Rectified Linear Unit** (ReLU):

$$\mathbf{x}_i^{(1)} = \text{ReLU} \left(\text{SAGEConv}^{(1)} \left(\mathbf{x}_i^{(0)}, \mathcal{N}(i) \right) \right) \quad (7)$$

dove $\mathcal{N}(i)$ rappresenta il vicinato del nodo i e $\mathbf{x}_i^{(1)} \in \mathbb{R}^{64}$.

3. **Secondo Strato SAGEConv:** Il secondo strato applica la medesima operazione, mantenendo la dimensionalità dell'incorporamento a 64 e permettendo alla rete di catturare informazioni fino a una distanza di 2 salti. Anche in questo caso viene applicata una funzione ReLU:

$$\mathbf{x}_i^{(2)} = \text{ReLU} \left(\text{SAGEConv}^{(2)} \left(\mathbf{x}_i^{(1)}, \mathcal{N}(i) \right) \right) \quad (8)$$

con $\mathbf{x}_i^{(2)} \in \mathbb{R}^{64}$.

4. **Layer Lineare (Readout):** L'embedding finale del nodo viene passato a uno strato lineare denso che riduce la dimensionalità da 64 al numero di colori target c . L'output di questo strato costituisce i logit grezzi $\mathbf{z}_i \in \mathbb{R}^c$ per il nodo i :

$$\mathbf{z}_i = W \cdot \mathbf{x}_i^{(2)} + \mathbf{b} \quad (9)$$

5. **Output:** Per ottenere la rappresentazione "soft" e continua dell'assegnazione dei colori, ai logit viene applicata la funzione **Softmax**. Questo garantisce che per ogni nodo i , l'output p_i sia una valida distribuzione di probabilità sui c colori disponibili:

$$\mathbf{p}_i = \text{Softmax}(\mathbf{z}_i) \quad (10)$$

La scelta di limitare la profondità a due soli strati di **message passing** previene il fenomeno dell'**over-smoothing** e garantisce che l'ottimizzazione dei parametri (focalizzata sulla minimizzazione della singola istanza) risulti stabile e rapida. Nel caso di grafi di dimensioni maggiori rispetto a quelli da noi presi in considerazione può risultare utile aggiungere uno strato. Riportiamo in Fig (1), lo schema dell'architettura.

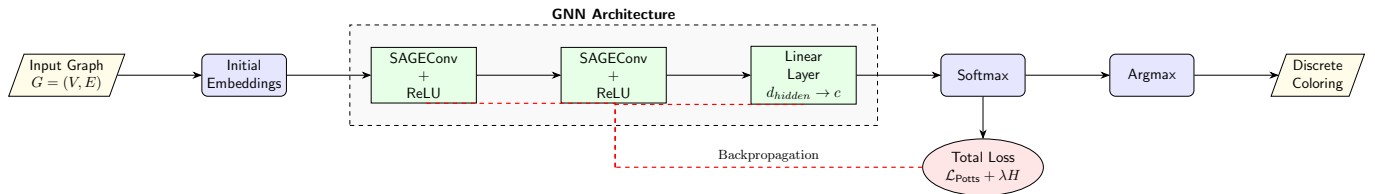


Figura 1: Architettura del modello.

Training

Come precedentemente discusso, l'addestramento non ha lo scopo di produrre un modello capace di generalizzare su dati non visti, bensì funge da vero e proprio algoritmo di ottimizzazione per la singola istanza del grafo. I parametri della GNN vengono inizializzati casualmente e aggiornati iterativamente al fine di minimizzare l'energia della configurazione.

L'ottimizzazione è stata condotta utilizzando l'algoritmo **Adam**. Durante la fase di sperimentazione iniziale, sono stati fissati i seguenti iperparametri:

- **Learning Rate (η):** Fissato a 0.003, un valore che garantisce una discesa rapida del gradiente limitando al contempo le oscillazioni eccessive attorno ai minimi locali. Non abbiamo ritenuto necessario l'uso di una schedula.
- **Epoche massime e pazienza:** Il limite superiore è stato impostato a 1000 epoche per run, fornendo alla rete un tempo sufficiente per raggiungere la convergenza energetica. Inoltre, in assenza di miglioramenti nella loss in 250 epoche, il training loop viene interrotto, così da garantire tempi ridotti.
- **Regolarizzazione Entropica (λ):** Il peso del termine entropico è stato modulato durante il training tramite una schedula:

$$\lambda(t = \text{epoch}) = \lambda_0 t / \tau, \quad (11)$$

con $\lambda_0 = 0.005$ e $\tau = 200$. In questo modo la rete è incoraggiata a prendere decisioni nette sulle assegnazioni di colore, senza però dominare il gradiente guidato dalla loss di Potts nelle prime fasi di addestramento.

Durante il training loop, sono state valutate costantemente due metriche complementari per valutare l'avanzamento della ricerca:

1. **Loss Continua (Energia):** Il valore quantitativo di $\mathcal{L}_{\text{tot}} = \mathcal{L}_{\text{Potts}} + \lambda H$ calcolato sulle probabilità. Costituisce l'unico segnale utilizzato dall'ottimizzatore per calcolare i gradienti tramite **backpropagation** e aggiornare i pesi.
2. **Conflitti Discreti (Validità Combinatoria):** Ad ogni epoca, in parallelo al calcolo della loss, l'output della rete subisce una discretizzazione tramite l'operatore **argmax**. Viene quindi contato il numero

effettivo di archi in conflitto ($N_{\text{conflicts}}$). Pur non essendo differenziabile, questa metrica rappresenta il vero indicatore di successo del task.

Le due curve mostrano andamenti correlati ma topologicamente distinti: la funzione di costo diminuisce in maniera continua, mentre il numero di conflitti si riduce a gradini. Spesso, variazioni significative nella loss continua precedono di alcune epoche la risoluzione effettiva di un conflitto nello spazio discreto.

Il panorama energetico generato dall'hamiltoniana di Potts è notoriamente non convesso e costellato da minimi locali. Durante l'addestramento, accade di frequente che la rete converga in configurazioni sub-ottimali, caratterizzate da un numero minimo di conflitti (es. 1 o 2) da cui la pura discesa del gradiente non riesce a fuggire a causa di barriere energetiche sfavorevoli.

Per rendere robusta la procedura, sono state implementate le seguenti logiche di controllo:

- **Early Stopping:** L'obiettivo della rete non è minimizzare asintoticamente la loss, ma trovare una validazione combinatoria. Se, a una qualsiasi epoca, si registra $N_{\text{conflicts}} = 0$, il training loop viene interrotto immediatamente con successo.
- **Post-Processing Locale:** In caso di stagnazione su un numero estremamente esiguo di conflitti residui al termine delle epoche (o della pazienza), prima di ricorrere al **multi-start** viene attivata la routine di ricerca locale **greedy** per tentare di alterare il colore dei soli nodi critici.
- **Strategia Multi-Start:** Qualora si finisca il training loop senza azzerare i conflitti e il **post-processing** risulti inefficace, si riprova ad ottimizzare la rete, inizializzando i pesi con un seed differente, forzandola così ad esplorare una nuova regione dello spazio dei parametri.

Ripetendo la procedura illustrata al variare del numero di colori disponibili c (partendo dal basso o dall'alto), si riesce a stimare il numero cromatico del grafo $\chi(G)$.

Risultati

Per valutare l'efficacia del framework proposto (implementato in **Python** sfruttando **PyTorch**), la pipeline è stata applicata a due grafi di test forniti nel formato standard DIMACS. Al fine di fornire un termine di paragone per le



prestazioni della GNN, i medesimi grafi sono stati processati utilizzando un classico algoritmo di colorazione **greedy** (implementato in **NetworkX**). I grafi studiati hanno le seguenti caratteristiche:

Grafo	Nodi	Archì	Stima greedy di $\chi(G)$
4	23	71	5
18	128	1170	20

Riportiamo in Fig. (2) e (3) il Grafo 4, con la relativa colorazione **greedy**.

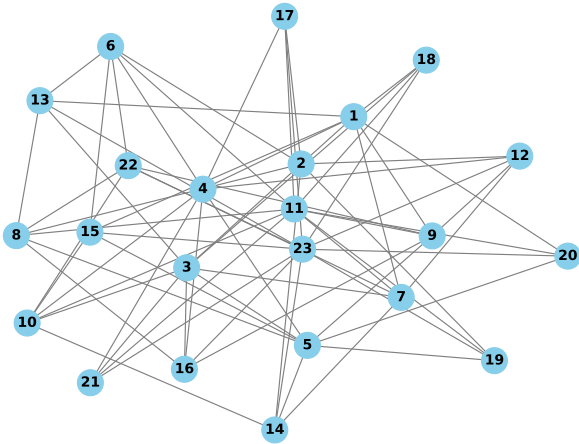


Figura 2: Grafo 4. Alcuni nodi potrebbero sembrare legati quando in verità non lo sono; ci scusiamo ma non è facile rappresentare grafi con molti nodi.

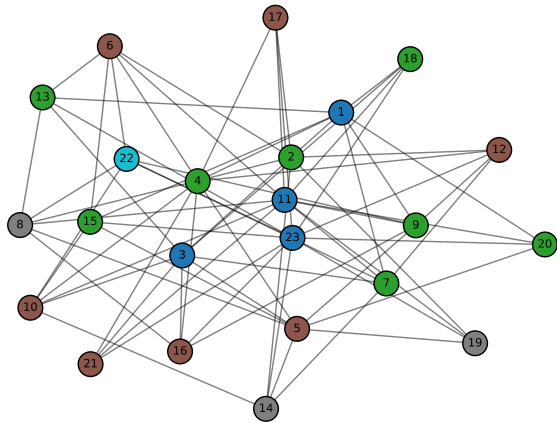


Figura 3: Colorazione del Grafo 4 tramite algoritmo **greedy**. Il numero cromatico stimato è 5.

L'analisi del processo di ottimizzazione per una singola istanza (con il numero di colori c fissato a un valore critico) rivela come la GNN esplori il panorama energetico. La Fig. (4) mostra l'evoluzione congiunta della loss totalee i contributi singoli, oltre che i conflitti discreti, in funzione del numero di epoche.

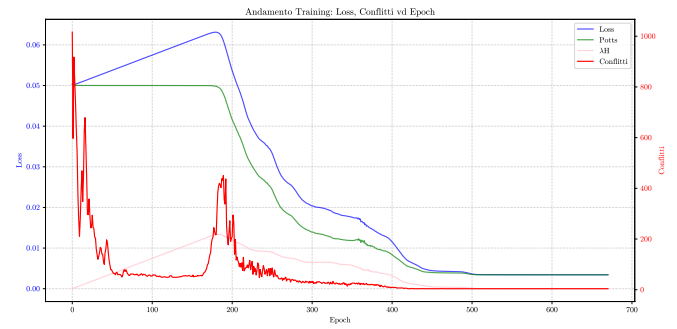


Figura 4: Training del Grafo 18 con 20 colori. Si nota una prima fase di esplorazione fino a circa 200 epoche, insieme ad una crescita del termine entropico. Successivamente vi è una decrescita contemporanea della loss di Potts, dell'entropia e del numero di conflitti, che suggerisce che il sistema si stia finalmente dirigendo verso una soluzione sub-ottimale, in cui rimane incastrato: infatti l'entropia va a zero ma la loss di Potts diventa costante. Tramite il post-processing si riuscirà facilmente a concludere il lavoro.

Come si evince dal grafico, la GNN riduce l'energia del sistema ma non sempre, contemporaneamente, il numero di conflitti discreti. In assenza di regolarizzazione entropica, per esempio, la loss decresce ma in alcuni casi il numero di conflitti non la accompagna, visto che l'energia di Potts potrebbe prediligere vettori di probabilità uniformi. Analogamente, con una regolarizzazione entropica troppo spinta la loss potrebbe obbligare i nodi ad assumere un colore ben definito ma non corretto, portando anche in questo caso ad una diminuzione della loss ma non dei conflitti.

Per stimare il numero cromatico $\chi(G)$, abbiamo seguito la strategia descritta in **Training**. Riportiamo in Fig. (5) l'andamento del numero di conflitti in funzione dei colori disponibili per il Grafo 18.

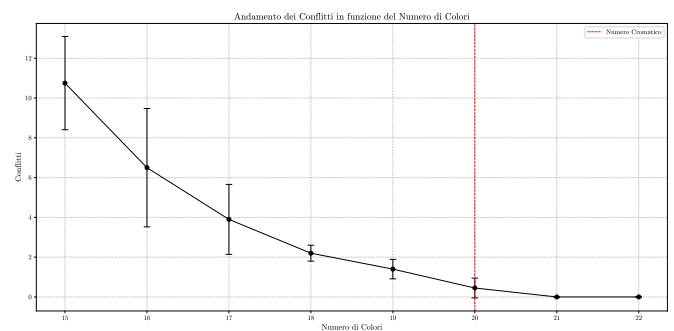


Figura 5: Il grafico mostra l'andamento dei conflitti medi, con relativa deviazione standard, su 20 run per colore, con post-processing se i conflitti risultavano minori di 10. Il numero cromatico stimato per il Grafo 18 è 20; nel grafico corrisponde ad un punto di transizione.

La Fig. (5) evidenzia chiaramente il punto di transizione: il numero medio di conflitti diminuisce all'aumentare dei gradi di libertà (colori). Il primo valore di c per cui



la rete riesce sistematicamente a trovare una soluzione a zero conflitti viene considerato come il numero cromatico stimato.

I risultati ottenuti tramite la GNN sono equivalenti a quelli dell'algoritmo **greedy** ($\chi(G_4) = 5$ e $\chi(G_{18}) = 20$), seppur i tempi di esecuzione sono maggiori. L'algoritmo **greedy** impiega meno di un millisecondo, rispetto alla rete che invece richiede per una singola esecuzione da centesimi di secondo fino a circa 5 secondi per una singola esecuzione, dipendentemente dalla dimensione del grafo, e dal qualche secondo fino a un minuto per la stima del numero cromatico.

A completamento dell'analisi, la configurazione ottimale estratta per il Grafo 4 è stata ricostruita visivamente in Fig. (6). La colorazione finale non presenta alcun arco monocromatico, confermando la correttezza formale della soluzione generata al termine del processo di discretizzazione.

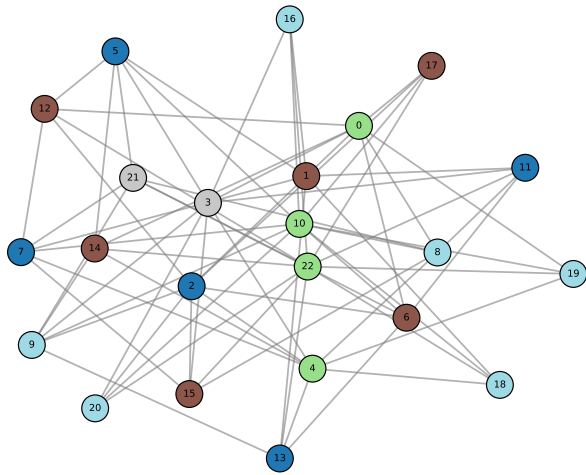


Figura 6: Colorazione del Grafo 4 tramite la GNN. Anche in questo caso il numero cromatico stimato è 5.

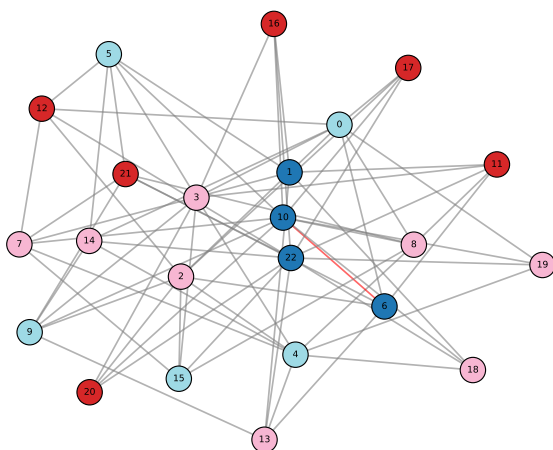


Figura 7: Esempio di colorazione incompleta per il Grafo 4. Con soli 4 colori non si è riusciti a colorare il grafo. In rosso l'arco conflittuale.

Invece in Fig. (7) riportiamo il caso di una colorazione con conflitti, nel caso di colori insufficienti.

Conclusioni

In questo elaborato è stato presentato e validato un framework basato su Graph Neural Network per la risoluzione del problema della colorazione di grafi attraverso una riformulazione differenziabile e continua. L'integrazione di una funzione di costo ispirata al modello di Potts con una regolarizzazione entropica ha permesso di guidare efficacemente la rete verso configurazioni nette e prive di conflitti, fornendo una stima accurata del numero cromatico $\chi(G)$ sulle istanze analizzate. Tuttavia, non essendoci una garanzia di esplorazione esaustiva dello spazio dei parametri, l'ottimalità globale non può essere garantita a priori, analogamente a quanto avviene per i metodi greedy.

L'analisi sperimentale ha evidenziato la rilevanza cruciale della configurazione algoritmica su più livelli. In primo luogo, la scelta degli embedding iniziali dei nodi si è rivelata determinante: l'utilizzo di vettori della base canonica (**one-hot**) associati all'identità del nodo ha garantito una corretta convergenza, laddove metriche basate sulle proprietà topologiche locali (come il grado del nodo) conducevano la rete a bloccarsi in minimi locali privi di significato combinatorio. In secondo luogo, la natura fortemente non convessa del panorama energetico ha confermato la necessità di affiancare alla discesa del gradiente strategie robuste di **multi-start** e una routine di **post-processing** locale di tipo greedy. Non di rado, infatti, la rete convergeva in soluzioni non ottimali, mostrando una grande dipendenza dall'inizializzazione casuale. Queste strategie si sono rivelate indispensabili per superare le barriere energetiche e azzerare gli ultimi conflitti discreti residui. I risultati finali, in perfetto accordo con le stime fornite dall'algoritmo greedy di riferimento, dimostrano le concrete potenzialità dei metodi neurali geometrici applicati ai problemi di ottimizzazione combinatoria su singola istanza. Allo stesso tempo l'analisi condotta ha però messo in luce problemi concreti come la necessità di un attento tuning degli iperparametri, non sempre banale, e di un approccio ibrido, indispensabile per risolvere le ultime criticità discrete; è proprio in questo regime critico che la fluidità del rilassamento continuo mostra i suoi limiti, lasciando il passo a strategie ibride capaci di spezzare lo stallo causato dalle simmetrie e azzerare gli ultimi conflitti residui. Per superare questi limiti strutturali, l'attuale routine di post-processing greedy potrebbe essere evoluta verso una vera e propria integrazione nativa della dinamica neurale con algoritmi classici, dando vita a solutori ibridi adattivi capaci di coniugare la flessibilità dell'apprendimento automatico con l'affidabilità dei metodi combinatori tradizionali.